

MBAT Club Datathon 2026 — Participant Manual

MBAT Club

Table of contents

Welcome	5
How to use this manual	5
Before you arrive	6
Getting help on the day	6
1 Code of Conduct	7
2 Git + GitHub Desktop	9
2.1 The daily loop	9
2.2 Install (before the event)	9
2.3 Get your team's repository	9
2.4 Work session workflow	10
2.5 Avoid stepping on your teammates	10
2.6 Fix a merge conflict	10
2.7 Never commit these	11
2.8 Prefer the command line?	11
2.9 Before you submit (Day 2, 11:00 AM deadline)	11
3 Quarto	12
3.1 Anatomy of a .qmd file	12
3.2 Render your report	13
3.3 Set your output format	13
3.4 Build presentation slides (required from every team)	13
3.5 Set up PDF rendering (before the event)	14
3.6 Keep the report reproducible and inside the word limit	14
3.7 Learn more	15
4 Using the Starter Template	16
4.1 Folder structure	16
4.2 Get set up	16
4.3 Use your script from a .qmd	17
4.4 Fill in the report and slides	17
4.5 Render everything	17
4.6 Submit	17

I	Databases + MongoDB	18
5	NoSQL and MongoDB: An Introduction for Statisticians	19
5.1	1. Why This Tutorial Exists	19
5.2	2. What “NoSQL” Actually Means	20
5.3	3. The Core Idea: Documents Instead of Rows	20
5.3.1	A translation table for vocabulary	21
5.4	4. Why Embed Instead of Normalize?	21
5.5	5. Schema Flexibility: A Blessing and a Curse	22
5.6	6. Querying: <code>dplyr</code> /SQL Habits vs. MongoDB’s Query Language	23
5.6.1	Comparison table	23
5.7	7. Using MongoDB from R	24
5.8	8. Consistency, Transactions, and Why This Matters to You	25
5.9	9. When Each Approach Tends to Fit Better	25
5.10	10. A Worked Mini-Example: Same Data, Both Ways	26
5.11	11. Summary	27
6	MongoDB Atlas to R: Connecting, Pipelines, and Text Analysis	28
6.1	What you’ll need	28
6.2	Know your collections before you query	29
6.3	Connecting to the cluster	30
6.4	The core mental model: pipelines are JSON, R is the courier	31
6.5	Don’t download the whole collection	31
6.5.1	Filter rows on the server with <code>query</code>	32
6.5.2	Let <code>\$group</code> summarize before it ever reaches R	33
6.5.3	Count before you fetch	33
6.5.4	<code>limit</code> , <code>skip</code> , and <code>\$sample</code> for peeking or paging	34
6.5.5	Stream through a large result with <code>\$iterate()</code>	34
6.5.6	A decision guide	34
6.6	A worked pipeline, stage by stage: joining reports to incidents	35
6.6.1	Stage 1 — <code>\$lookup</code> (a SQL-style JOIN)	36
6.6.2	Stage 2 — <code>\$unwind</code> (flatten the array)	36
6.6.3	Stage 3 — <code>\$project</code> (reshape the output)	36
6.7	The general thinking process for writing a new pipeline	37
6.7.1	Worked example: <code>\$match</code> + <code>\$group</code> from scratch	38
6.8	Two ways to join <code>reports</code> to <code>incidents</code> — pipeline vs. R-side	39
6.9	Cleaning the text	40
6.10	Tokenising and analysing with <code>tidytext</code> — size-consciously	40
6.10.1	Word frequencies	41
6.10.2	TF-IDF (which words distinguish one incident/domain from another)	41
6.10.3	N-grams	42
6.10.4	Summarize text metrics server-side instead of tokenizing in R	42
6.10.5	Process the full corpus in chunks, when you must	43

6.11	Sentiment analysis	43
6.12	Word co-occurrence / correlation (optional, <code>widyr</code>)	44
6.13	Debugging and building pipelines incrementally	45
6.14	Putting it together: a minimal end-to-end script	45
6.15	Common pitfalls	46
6.16	Quick reference	47
6.16.1	Pipeline stages	47
6.16.2	Package cheat sheet	47
6.17	Summary checklists	48
7	Data dictionary for AI Incident Database	49
7.1	Overview	49
7.2	Core record collections	49
7.2.1	<code>incidents</code>	49
7.2.2	<code>reports</code>	51
7.2.3	<code>entities</code>	52
7.2.4	<code>entity_relationships</code>	52
7.3	Classification (taxonomy) collections	53
7.3.1	<code>taxa</code>	53
7.3.2	<code>classifications</code>	54
7.4	Editorial pipeline & workflow collections	55
7.4.1	<code>submissions</code>	55
7.4.2	<code>candidates</code>	56
7.4.3	<code>quickadd</code>	57
7.4.4	<code>duplicates</code>	57
7.4.5	<code>entity_duplicates</code>	57
7.4.6	<code>incident_links</code>	58
7.4.7	<code>checklists</code>	58
7.4.8	<code>migrations</code>	59
7.5	Notes on schema conventions	59
8	Resources for tidytext (Text Mining with R)	61
8.1	The core reference	61
8.2	If you want to find latent themes: topic modeling (LDA)	61
8.3	If word variants are diluting your counts: <code>stemming</code>	62
8.4	If you want more corpus-linguistics tooling: <code>quanteda</code>	62
8.5	Visualizing results	62
8.6	Cheat sheet	62
8.7	Keep it in scope	63
9	Good Visualisation & Tables	64

Welcome

This is the participant manual for the **MBAt Club Datathon 2026** (Econometrics and Business Statistics), held Monday 21 – Tuesday 22 September at Learning and Teaching Building, Clayton campus.

It's a reference to work through **before** the event and to keep open **during** the event — you're not expected to read it cover to cover in one sitting, but every chapter should make sense as a standalone lookup when you're stuck on something specific.

 Never push API keys or credentials to GitHub

Your team's repository is **public**. Your MongoDB connection string (and any other password or API key) must never appear in a committed file — keep it in a local `.Renviron` that stays out of the repo. See Chapter 2 and Chapter 6 for the full setup.

How to use this manual

- Chapter 1 — the standards of behaviour we expect from everyone at the event, and how to report a concern.
- Chapter 2 — your team's submission is a public GitHub repository, so this is the one chapter everyone on the team should be comfortable with before Day 1.
- Chapter 3 — how to write and render the reproducible report and the presentation slides your submission requires — both are mandatory, not just for finalists.
- Chapter 4 — the folder layout your team's repository starts from, and how to fill in and render it.
- Chapter 5 and Chapter 6 — the competition dataset lives in a MongoDB Atlas cluster; these two chapters cover the concepts and the hands-on connection guide for pulling it into R.
- Chapter 8 — further reading and techniques for text mining the (mostly free-text) dataset, beyond what's already worked through in the MongoDB chapter.
- Chapter 9 — turning your analysis into a figure or table a judge can actually read at a glance: choosing the right chart, designing for perception, and building a communicative table from a fitted model.

Before you arrive

- Install R, RStudio (or Positron/VS Code with the R extension), Quarto, and GitHub Desktop.
- Create a GitHub account if you don't already have one, and share your username with organisers — they create and share your team's repository, so you just need to clone it (Chapter 2).
- Skim Chapter 2 and Chapter 3 ahead of time — Day 1 morning is registration, opening remarks, and the dataset introduction, and the working session doesn't start until 10:30 AM, so setup issues are much cheaper to fix beforehand than mid-competition.

Getting help on the day

A help desk is staffed for the full duration of both days. If you're stuck on something in this manual for more than a few minutes, ask rather than losing working time to it.

1 Code of Conduct

We are committed to providing a safe, inclusive, welcoming, and harassment-free experience for everyone.

We try to cultivate a community with shared values, where people are comfortable exploring ideas, asking questions, and saying things like “I don’t understand” or “why”. Assume competence in the people you interact with. There are no stupid questions.

Be considerate in speech and actions, and actively seek to acknowledge and respect the boundaries of fellow participants.

Take care of each other. Alert a member of the Code of Conduct Committee (named below) if you notice a dangerous situation, someone in distress, or a potential violation of this Code of Conduct, even if it seems inconsequential.

We do not tolerate harassment in any form. Refrain from demeaning, discriminatory, or harassing behavior and speech. Harassment includes, but is not limited to:

- offensive comments related to gender, gender identity and expression, age, sexual orientation, disability, physical appearance, body size, race, ethnicity, religion (or lack thereof), or technology choices
- unwelcome comments regarding a person’s lifestyle choices and practices, including those related to food, health, parenting, drugs, or employment
- sustained or willful disruption of discussions, talks, or other events
- deliberate intimidation
- continued one-on-one communication after requests to cease
- unwanted photography or recording, including logging online activity for harassment purposes
- pattern of inappropriate social contact, such as requesting/assuming inappropriate levels of intimacy with others
- inappropriate physical contact without consent or after a request to stop
- stalking or following
- threats of violence or incitement of violence towards any individual, including encouraging a person to commit suicide or to engage in self-harm
- use of sexual or discriminatory imagery, comments, or jokes
- unwelcome sexual attention
- deliberate “outing” of any aspect of a person’s identity without their consent except as necessary to protect vulnerable people from intentional abuse

- deliberate misgendering or use of “dead” or rejected names
- publication of non-harassing private communication

This Code of Conduct applies to all people participating in the event, including staff and leadership. It applies to all modes of interaction online including GitHub project repositories, including social gatherings affiliated with the event.

Reporting Guidelines The Code of Conduct Committee includes

- Krisanat Anukarnsakulchularp
- Janith Wanniarachchi
- Maliny Po

If you experience or witness unacceptable behavior, or have any other concerns, please submit a report to the Committee as soon as possible on ***mbat-eps@monash.edu***. You may also make a report directly to one member of the Committee by contacting them directly. If any member of the Committee has a conflict of interest with a report, they will be recused and will not have access to the content or process of the report followup.

The Code of Conduct Committee will investigate and decide responses to reports with the aim of making a decision and implementing enforcement as soon as is reasonably possible. We are committed to transparency with our community while upholding the privacy of victims.

We welcome your feedback and thank you for working with us to make rOpenSci a safe, enjoyable, friendly and enriching experience for everyone who participates.

This Code of Conduct is adapted from those used by ropensci.org and OceaniaR Hackathon 2025

2 Git + GitHub Desktop

Why this matters

Your submission is a public GitHub repository. A team that can't reliably commit and push loses time to lost work, overwritten files, or a repo that isn't actually up to date at the deadline.

2.1 The daily loop

Everything below is a variation on this:

1. **Pull** the latest changes.
2. Do some work.
3. **Commit** with a short message.
4. **Push** so your team can see it.

2.2 Install (before the event)

1. Install [GitHub Desktop](#).
2. Sign in with a GitHub account (create one first if needed).
3. Let it install Git automatically if prompted.
4. Make sure organisers have your GitHub username so you can be added to your team's repository.

2.3 Get your team's repository

Organisers create and share one repository per team, already set up from the starter template (Chapter 4) — you don't create anything.

1. Get the repo link from organisers.
2. Every team member: GitHub Desktop → **File** → **Clone Repository** → paste the URL → **Clone**.

3. Before the working session starts, confirm you can push: make a trivial commit (e.g. add your name to the README) and push it.

2.4 Work session workflow

1. Click **Fetch origin**, then **Pull origin** if you're behind — do this *before* you edit anything.
2. Edit your files as normal.
3. Check the **Changes** tab — untick anything you didn't mean to include.
4. Write a short summary → **Commit to main**.
5. Click **Push origin**.
6. Repeat: pull → work → commit → push.

Commit often — every working chunk of progress, not just once at the end of the day.

2.5 Avoid stepping on your teammates

- Split work by file/section so two people aren't editing the same lines at once.
- Pull before you start a session; push before you take a break.
- Say out loud (or in chat) before editing a shared file, e.g. the report.

2.6 Fix a merge conflict

1. Open the file GitHub Desktop flags as conflicted.
2. Find the marked sections:

```
<<<<<<< HEAD
your version
=====
their version
>>>>>>> branch-name
```
3. Edit the file to keep the correct content (or merge both).
4. Delete the <<<<<<<, =====, >>>>>>> marker lines.
5. Save, then commit the resolved file as usual.
6. Stuck for more than a few minutes? Ask the help desk.

2.7 Never commit these

- **Credentials** — MongoDB connection string, API keys, `.Renviron` contents. Keep them in a local `.Renviron` file, listed in `.gitignore`.
- **Large raw data dumps** — pull from the database at analysis time instead.
- **Cache/temp folders** — `.quarto/`, `*_cache/`. (`_docs/` is the exception — see Chapter 4: that one *does* need to be pushed, since it's your submitted report and slides.)

If a credential is ever pushed by accident, rotate it immediately — deleting the file does not remove it from the repo's history.

2.8 Prefer the command line?

```
git clone <repo-url>
git status
git add <file>          # or `git add .`
git commit -m "message"
git pull
git push
```

Same repository either way — teammates can mix GitHub Desktop and the terminal freely.

2.9 Before you submit (Day 2, 11:00 AM deadline)

- ☐ The final commit is pushed — an unpushed commit does not count as submitted
- ☐ No credentials appear anywhere in the repo, including old commits
- ☐ Both the rendered HTML and PDF reports open correctly from a fresh clone
- ☐ The presentation slides (Chapter 3) are finished and rendered — required from every team, not just finalists

3 Quarto

i Why this matters

Your submission requires both a reproducible report and a set of presentation slides — Quarto builds both from the same kind of source file. It runs your R code and inserts the real output every time you render, so you never copy-paste a number or a plot in by hand.

The starter template (Chapter 4) already includes both: `docs/report.qmd` for the written report and `docs/slides.qmd` for presentation slides.

3.1 Anatomy of a .qmd file

```
---
title: "Team Name - Datathon Submission"
author: "Team Name"
format: html
---

# Introduction

We analyse the AI Incident Database to answer...

```{r}
library(tidyverse)
summary(cars)
```
```

- Text outside code chunks is normal Markdown.
- Anything inside ````{r} ```` blocks runs at render time.
- Add `#| echo: false` inside a chunk to keep the output but hide the code — useful for staying within a word limit.

3.2 Render your report

- RStudio/Positron: click **Render**, or press Cmd/Ctrl+Shift+K.
- Command line:

```
quarto render report.qmd      # render using the format in the YAML
quarto render report.qmd --to pdf # force a specific format
quarto preview report.qmd      # live preview, reloads on save
```

Use `quarto preview` while you write — it catches broken chunks immediately instead of at the end.

3.3 Set your output format

Both HTML and PDF are required for the report — list both formats in the YAML and Quarto renders one file for each:

```
format:
  html:
    toc: true
    code-fold: true # readers can expand/collapse code blocks
  pdf:
    documentclass: article
```

```
quarto render report.qmd # produces report.html and report.pdf
```

The report has a **3,000-word limit**, excluding code, tables, figure captions, and references.

3.4 Build presentation slides (required from every team)

A finished slide deck is part of your submission, not an optional extra — every team builds one, even though only the 5 finalists actually present it live (2:15–3:30 PM, Day 2: 10 min talk + 5 min Q&A each). Treat it with the same care as the report: it's judged on Professional Communication just like everything else you submit.

```
---
title: "Team Name - Datathon Presentation"
format: revealjs
---
```

1. Start each new slide with a `##` heading.
2. Write R chunks exactly as you would in the report — plots and tables render live into the deck.
3. Render the same way: **Render** button, `Cmd/Ctrl+Shift+K`, or `quarto render slides.qmd`.
4. Open the resulting `.html` in a browser; press `f` for fullscreen, use arrow keys or space to advance.

```
## Introduction

The question we investigated...

## Our Approach

```{r}
ggplot(data, aes(x, y)) + geom_point()
```

## Results

Key finding...
```

3.5 Set up PDF rendering (before the event)

Since PDF is a required output, don't leave this until Day 2. PDF needs LaTeX — install it once, ahead of time:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Render both formats and open both files before the deadline — a report that renders fine to HTML can still fail on PDF (e.g. an image or table that doesn't fit the page).

3.6 Keep the report reproducible and inside the word limit

1. Render early and often — don't wait until the deadline to find a broken chunk.
2. Decide how much code you want visible in the final report — the `echo`, `warning`, and `message` options (set per chunk, or globally under `execute:` in the YAML) control this. There's no required setup here; choose whatever makes your report read cleanly.

3. Cache slow steps (a MongoDB pull, a model fit) with `#| cache: true` on that chunk.
4. Use `knitr::kable()` or `gt` for tables, and pipe large printed output through `head()` — raw tibble/model printouts can quietly blow out your word count.
5. Keep the raw `.qmd` in your GitHub repo, not just the rendered file — the repo is what gets checked for reproducibility.

3.7 Learn more

- `<starttr.numbat.space>` — the club’s starting point for Quarto.
- <https://quarto.org/docs/guide/> — official docs, every format and option.
- `quarto --help` on the command line, or `?quarto::quarto_render` in R.

Stuck on a YAML typo for more than a few minutes? Ask the help desk.

4 Using the Starter Template

i Why this matters

The template gives every team the same layout and a couple of pieces (a database-connection stub, report/slide skeletons) already wired up, so you spend Day 1 morning on data, not project setup.

4.1 Folder structure

- `src/` — your R script files. Ships with `main.R`, which stubs out a `connect_to_db()` function for you to implement.
- `data/` — intermediate computed datasets you save as you go.
- `docs/` — your Quarto files: `report.qmd` (written report) and `slides.qmd` (presentation) — see Chapter 3.
- `_docs/` — generated automatically when you render (set as the `output-dir` in `_quarto.yml`). This holds your final rendered report and slides — commit and push it, since it's what organisers see.
- `manual/` — reference material from organisers.

Following this layout isn't mandatory, but it's what lets organisers find your work quickly — worth it for the Professional Communication judging pillar.

4.2 Get set up

1. Clone your team's repo (Chapter 2).
2. Open it in RStudio/Positron.
3. Install the packages the template uses: `dplyr`, `mongolite`, [here](#).
4. Open `src/main.R` and implement `connect_to_db()` using your Atlas connection string (Chapter 6) — read it with `Sys.getenv()`, never hardcode it.

4.3 Use your script from a .qmd

Don't copy-paste code between `src/main.R` and your `.qmd` files — source it instead. `docs/report.qmd` already does this at the top:

```
source(here::here("src/main.R"))
```

`here::here()` keeps file paths working no matter which file you render from — always reference files this way rather than relative paths like `"../src/main.R"`.

For more selective imports, `box::use()` is also available if you're already familiar with it — optional, not required.

4.4 Fill in the report and slides

1. `docs/report.qmd` — starts as `format: html` only. Both HTML **and** PDF are required, so add a `pdf:` entry to the YAML before you submit (Chapter 3 shows the format block and PDF setup). Set your team name in the title too.
2. `docs/slides.qmd` — already set to `format: revealjs`. A finished deck is required from every team, not just finalists — only the 5 finalists present it live, but everyone submits one (Chapter 3 covers building slides).

4.5 Render everything

From the project root:

```
quarto render
```

This renders every `.qmd` under `docs/` and writes the output into `_docs/`.

4.6 Submit

There's no file to zip or upload — organisers submit by cloning your repository directly.

1. Run `quarto render` so `_docs/` is up to date.
2. Commit and push everything, including `_docs/` (Chapter 2) — unlike a typical build folder, this one needs to be pushed, since it's your actual submitted report and slides.
3. Confirm the repo is public and the final push landed before the deadline.

Part I

Databases + MongoDB

5 NoSQL and MongoDB: An Introduction for Statisticians

i Why this matters

The competition data lives in MongoDB, not tables. If your mental model is still “rows and joins,” a few of MongoDB’s habits will trip you up — this chapter re-orientates that model before you touch the real connection in Chapter 6.

Already comfortable with SQL vs. NoSQL? Skip straight to Chapter 6 and test your connection before the event starts — see its “Common pitfalls” section on network/allow-listing issues.

Written for readers comfortable with R, data frames, and relational databases (SQL, normalized tables, joins).

5.1 1. Why This Tutorial Exists

If you’ve spent time with relational databases, your mental model of data storage is probably something like this:

- Data lives in **tables** with a **fixed schema** (columns defined in advance, with types).
- Rows relate to each other through **foreign keys**, and you stitch related data back together with **joins**.
- You enforce consistency using constraints (primary keys, foreign keys, NOT NULL, etc.).
- A single logical “thing” — like a customer with multiple orders — is normalized across several tables to avoid duplication.

This model maps naturally onto how you already think about data as a statistician: a rectangular data frame, one row per observation, one column per variable, `merge()` or `dplyr::left_join()` to combine data frames.

NoSQL databases (of which MongoDB is the most popular example) relax or abandon several of these assumptions. Understanding *why* they do this, and *what you gain and lose*, is the goal of this tutorial.

5.2 2. What “NoSQL” Actually Means

“NoSQL” is a loose umbrella term (sometimes glossed as “Not Only SQL”) covering several different data models that don’t follow the relational (table-and-join) paradigm. The major families are:

| Type | Data model | Example systems |
|-------------|---------------------------------------|------------------|
| Document | Semi-structured documents (JSON-like) | MongoDB, CouchDB |
| Key-value | Simple key → value lookups | Redis, DynamoDB |
| Wide-column | Sparse, column-family tables | Cassandra, HBase |
| Graph | Nodes and edges | Neo4j, ArangoDB |

This tutorial focuses on **document stores**, and specifically **MongoDB**, because they’re the most common entry point and the comparison to relational tables is the most direct.

5.3 3. The Core Idea: Documents Instead of Rows

 The data used in this example

The examples used in this page are not reflective of the actual data you will use in the competition! They are only presented to get an understanding of the functionalities.

In a relational database, a customer and their orders might be split across two tables:

```
-- customers table
customer_id	name	email
1           | Alice   | alice@email.com

-- orders table
order_id	customer_id	item	amount
101      | 1           | Widget | 9.99
102      | 1           | Gadget | 19.99
```

To get “Alice and all her orders,” you JOIN on `customer_id`.

In MongoDB, you’d likely store this as a **single document**:

```
{
  "_id": "1",
  "name": "Alice",
  "email": "alice@email.com",
  "orders": [
    { "item": "Widget", "amount": 9.99 },
    { "item": "Gadget", "amount": 19.99 }
  ]
}
```

A “document” is essentially a JSON object (technically stored as **BSON**, a binary superset of JSON with extra types like dates and integers). Documents are grouped into **collections**, which are the loose analogue of tables — except a collection has no fixed schema. Two documents in the same collection can have completely different fields.

5.3.1 A translation table for vocabulary

| Relational term | MongoDB term |
|-----------------|---|
| Database | Database |
| Table | Collection |
| Row | Document |
| Column | Field |
| Primary key | <code>_id</code> field |
| Join | Embedding, or <code>\$lookup</code> (an actual join-like operation) |
| Schema (rigid) | Schema (optional / flexible) |

5.4 4. Why Embed Instead of Normalize?

This is the conceptual heart of the difference, and it’s worth dwelling on because it inverts a habit that’s deeply ingrained if you’ve learned relational design.

Relational design philosophy: normalize aggressively. Store each fact once. Avoid redundancy. Joins are cheap(-ish) and consistency matters more than read speed.

Document design philosophy: model around your *access patterns*. If you almost always fetch a customer along with their orders together, store them together, even if that means the word “Widget” is duplicated across many documents. Reads become a single lookup instead of a join across multiple tables — this matters a lot for web-application workloads with high read volume and simple, predictable query shapes.

The tradeoff: - **Relational:** harder to scale horizontally, but strong guarantees around consistency and no duplicated data to keep in sync. - **Document:** easier to scale horizontally and often faster for “fetch this one entity and everything about it” queries, but you may duplicate data and must manage the consequences (e.g., if Alice changes her email, and it’s embedded in old order documents, you may need to update it in multiple places, or decide you don’t care).

There’s no free lunch here — it’s an explicit modeling decision, not a strictly-better technology.

5.5 5. Schema Flexibility: A Blessing and a Curse

In a relational database, adding a column to a 10-million-row table requires an `ALTER TABLE` migration and some care. In MongoDB, you can insert a document with a new field at any time — the collection doesn’t complain.

For a statistician, the closest analogy is the difference between:

- An R `data.frame`/tibble with fixed columns (relational table), versus
- A list of lists, where each element can have different named entries (MongoDB collection).

```
# Relational-style: a data.frame, fixed columns
df <- data.frame(id = 1:2, name = c("Alice", "Bob"), age = c(30, NA))

# Document-style: a list of lists, heterogeneous structure
docs <- list(
  list(id = 1, name = "Alice", age = 30, hobbies = c("chess", "running")),
  list(id = 2, name = "Bob") # no age, no hobbies - and that's fine
)
```

This flexibility is genuinely useful when your data is naturally irregular (user-generated content, event logs with varying attributes, nested/hierarchical records). It’s a liability when you *want* the enforcement — e.g., you want the database itself to guarantee that `age` is always numeric, or that every document has an `email`. MongoDB does support optional **schema validation** rules, but they’re opt-in rather than the default posture of the system.

5.6 6. Querying: dplyr/SQL Habits vs. MongoDB's Query Language

MongoDB queries are expressed as documents themselves (essentially, nested lists/JSON), which feels unusual at first if you're used to SQL's `SELECT ... WHERE ...` syntax or `dplyr::filter()`.

5.6.1 Comparison table

| Task | SQL | dplyr | MongoDB (shell syntax) |
|-------------------|--|--|---|
| Filter rows | <code>SELECT * FROM orders WHERE amount > 10</code> | <code>filter(orders, amount > 10)</code> | <code>db.orders.find({ amount: { \$gt: 10 } })</code> |
| Select columns | <code>SELECT item, amount FROM orders</code> | <code>select(orders, item, amount)</code> | <code>db.orders.find({}, { item: 1, amount: 1 })</code> |
| Sort | <code>ORDER BY amount DESC</code> | <code>arrange(desc(amount))</code> | <code>.sort({ amount: -1 })</code> |
| Group + aggregate | <code>GROUP BY customer_id</code> | <code>group_by(customer_id) %>% summarise(...)</code> | <code>\$group</code> stage in an aggregation pipeline |

MongoDB's analogue to a SQL query with `GROUP BY`/joins/multiple transformations is the **aggregation pipeline** — a sequence of stages, each transforming the documents flowing through it, conceptually similar to chaining `dplyr` verbs with `%>%`:

```
db.orders.aggregate([
  { $match: { amount: { $gt: 5 } } },      // like filter()
  { $group: { _id: "$customer_id",
              total: { $sum: "$amount" } } }, // like group_by() %>% summarise()
  { $sort: { total: -1 } }                 // like arrange(desc())
])
```

If you're comfortable reading a `dplyr` pipe, the aggregation pipeline will feel more familiar than it first appears — it's a sequence of verbs applied in order, just expressed as JSON stage objects instead of R function calls.

5.7 7. Using MongoDB from R

You don't have to leave R to work with MongoDB. The `mongolite` package provides a clean interface, and the workflow will feel close to what you already do with DBI.

```
install.packages("mongolite")
library(mongolite)

# Connect to a collection (analogous to opening a DBI connection to a table)
orders <- mongo(collection = "orders", db = "shop", url = "mongodb://localhost")

# Insert documents (rows) - takes a data.frame directly
orders$insert(data.frame(item = "Widget", amount = 9.99, customer_id = 1))

# Query and get a data.frame back - this is the key convenience
result <- orders$find('{"amount": {"$gt": 5}}')
head(result)

# Aggregation pipeline, expressed as a JSON string
pipeline <- '[
  {"$group": {"_id": "$customer_id", "total": {"$sum": "$amount"}}},
  {"$sort": {"total": -1}}
]'
summary_df <- orders$aggregate(pipeline)
```

Compare this to a typical relational workflow with DBI:

```
library(DBI)
library(RPostgres)

con <- dbConnect(RPostgres::Postgres(), dbname = "shop")
result <- dbGetQuery(con, "SELECT * FROM orders WHERE amount > 5")
```

The big practical difference: `mongolite` queries and pipeline stages are written as **JSON strings**, not SQL strings — but the round trip (query in, tibble/data.frame out) is conceptually identical, which is good news for how much of your existing R workflow transfers.

5.8 8. Consistency, Transactions, and Why This Matters to You

Relational databases are traditionally built around **ACID** guarantees (Atomicity, Consistency, Isolation, Durability) — a transaction either fully happens or fully doesn't, and the database enforces referential integrity via constraints.

Historically, NoSQL document stores traded some of these guarantees for scalability, often described (loosely) via the **CAP theorem**: in the presence of a network partition across distributed servers, a system must choose between **Consistency** (every read gets the latest write) and **Availability** (every request gets *a* response, possibly stale). Many early NoSQL systems leaned toward availability and “eventual consistency.”

Modern MongoDB has narrowed this gap substantially — it now supports multi-document ACID transactions — but the *default* mode of use, and the ecosystem's design instincts, still favor flexible, denormalized, single-document operations over enforced cross-table integrity. If your work involves financial ledgers or anything where “the numbers must always add up, no exceptions, ever,” that's traditionally been relational-database territory, and it's worth understanding that this is a design lineage question, not just a feature checkbox — you can bolt strong consistency onto MongoDB, but it's swimming against the current of how the system is typically used.

5.9 9. When Each Approach Tends to Fit Better

| Favor relational (SQL) when... | Favor document (MongoDB) when... |
|--|---|
| Your data is naturally tabular and stable in structure | Your data is nested, hierarchical, or has varying fields per record |
| You need complex multi-table joins and strong referential integrity | You mostly fetch one entity plus its closely related data together |
| Strong consistency/transactions across entities is essential | High write/read throughput and horizontal scaling matter more |
| Your analysis workflow is fundamentally about tidy rectangular tables (which is common in statistics!) | Your data is closer to logs, events, user profiles, or content with irregular shape |

For a lot of statistical work — clean, rectangular datasets destined for modeling — relational databases (or just flat files/data frames) remain a very natural fit, and you'll often find yourself **flattening** MongoDB documents back into rectangular data frames before doing any actual analysis, since most statistical methods assume tidy, rectangular input.

5.10 10. A Worked Mini-Example: Same Data, Both Ways

Relational (two tables + a join):

```
CREATE TABLE customers (customer_id INT PRIMARY KEY, name TEXT);
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT REFERENCES customers(customer_id),
  item TEXT,
  amount NUMERIC
);

SELECT c.name, o.item, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

Document (one collection, embedded):

```
library(mongolite)
customers <- mongo("customers", db = "shop")

customers$insert('{
  "_id": 1,
  "name": "Alice",
  "orders": [
    {"item": "Widget", "amount": 9.99},
    {"item": "Gadget", "amount": 19.99}
  ]
}')

# Pull it back out, then flatten to a data.frame for analysis
raw <- customers$find()
flat <- tidyr::unnest(raw, orders)
```

Notice the last line: even when working with MongoDB, you often end up reaching for `tidyr::unnest()` to get back to the rectangular shape you actually want for modeling. This is a common and completely normal pattern — document stores are often optimized for how an *application* writes and reads data, not for how a *statistician* wants to analyze it, and translating between the two is a routine part of the workflow.

5.11 11. Summary

- MongoDB stores flexible, JSON-like **documents** in **collections**, instead of enforcing rows-in-tables with a fixed schema.
- The relational instinct to *normalize* is replaced by a document instinct to *denormalize/embed*, optimized around how the data will be read.
- Querying feels different syntactically (JSON-shaped queries, aggregation pipelines) but maps conceptually onto SQL/`dplyr` verbs you already know.
- Consistency and transactional guarantees are available in modern MongoDB but aren't the system's traditional default posture the way they are in relational databases.
- For statistical analysis specifically, you'll frequently pull data *out* of MongoDB and flatten it back into rectangular form — so document databases are often best understood as an operational/application-layer tool, with relational-style rectangular data as the destination for actual modeling.

If you want to go deeper next, worthwhile follow-ups are: MongoDB's aggregation pipeline in more depth, schema validation rules, and how sharding/replication work for horizontal scaling.

6 MongoDB Atlas to R: Connecting, Pipelines, and Text Analysis

Why this matters

You can't analyse a dataset until you can pull it out of Atlas and into a tidy data frame — and once you can, the next skill is knowing how to shape and summarize it *before* it crosses the network, so a few hundred thousand documents don't turn a five-minute script into a fight with your machine's memory. This sheet covers both: connecting and querying, writing and reasoning about aggregation pipelines, and doing tidyverse/text-mining work in a size-conscious way.

Tip

The starter template's `src/main.R` (Chapter 4) has an empty `connect_to_db()` function waiting for you — that's where the connection code below goes.

It's written against the AI Incident Database schema, so the field names below match what you'll actually see, but the *thinking process*, how to read a pipeline, how to build a new one from scratch, how to avoid downloading more than you need, carries over to any MongoDB collection you work with.

6.1 What you'll need

- R (4.x) and RStudio (or Positron / VS Code with the R extension)
- The connection string for your Atlas cluster (competition organisers will give you this; it looks like `mongodb+srv://<username>:<password>@ai-incident-database.ugkyb5n.mongodb.net/`)
- The database name is `aaidprod`

Install the packages you'll need once, up front:

```
install.packages(c(
  "mongolite",    # connect to MongoDB and pull data into R
  "tidyverse",    # dplyr, purrr, ggplot2, tidyr, etc.
  "tidytext",     # tokenisation, sentiment lexicons, tf-idf
  "jsonlite",     # handling nested JSON/BSON fields, building pipelines as lists
  "textclean",    # cleaning up messy text (contractions, HTML, etc.)
  "wordcloud2",   # optional, for word cloud visuals
  "widyR"         # optional, for word-pair / correlation analysis
))
```

`mongolite` is the simplest way to talk to MongoDB from R, and it hands data back as a data frame, which is exactly the shape tidyverse tools expect. Both `$find()` (simple queries) and `$aggregate()` (multi-stage pipelines) live on the connection object it gives you.

6.2 Know your collections before you query

The schema you’ve been given defines several collections. The two you’ll care about most are:

- **reports** — the actual documents/articles. This is where the free text lives. Key fields:
 - `_id` (objectId), `report_number` (number) — `report_number` is the human-friendly key used to link a report to the incident(s) it belongs to
 - `title`, `text`, `plain_text` — `text` may retain markdown/HTML formatting from the original source; `plain_text` is the cleaned version and is almost always what you want for tokenising/sentiment work
 - `description` — a short editor-written summary, separate from the report body
 - `authors`, `submitters` — arrays of strings
 - `tags` — array of free-text tags
 - `language`, `source_domain`, `url`
 - `date_published`, `date_submitted`, `date_downloaded`, `date_modified`
 - `embedding` — a precomputed vector embedding of the text (nested object with a `vector` field); large, and you generally won’t need it unless you’re doing similarity/clustering work directly on the provided vectors
 - `is_incident_report` (boolean)
- **incidents** — the higher-level “event” that one or more reports are describing. Key fields:
 - `_id` (objectId), `incident_id` (number)
 - `title`, `description`, `date`

- **reports** — an **array of report_number values** (not full documents) pointing back into the **reports** collection — this is how you join the two collections
- **Alleged deployer of AI system, Alleged developer of AI system, Alleged harmed or nearly harmed parties** — arrays of entity-name strings
- **implicated_systems** — array of strings
- **editors, editor_notes**
- **classifications** — tagging/label data (an **attributes** field) that links back to **incidents/reports** by their IDs — useful if you want to bring in categorical labels alongside the report text.
- **entities** — a lookup table of named entities (**entity_id, name**) referenced by the “alleged deployer/developer/harmed parties” fields above.

For most text-analysis tasks, you’ll mainly work with **reports.plain_text**, optionally joined to **incidents** for grouping/labels — which is exactly the join used as the worked example throughout this sheet.

6.3 Connecting to the cluster

Never hard-code your username and password directly into a script you might commit to GitHub or share with teammates. Use an **.Renviron** file instead.

Create/edit your .Renviron file (run this once):

```
usethis::edit_r_environ()
```

Add a line like this, save, and restart R:

```
MONGO_URI="mongodb+srv://myusername:mypassword@ai-incident-database.ugkyb5n.mongodb.net/"
```

Now connect from your script — one connection object per collection:

```
library(mongolite)

db_name <- "aiidprod"

reports_conn <- mongo(collection = "reports", db = db_name,
                      url = Sys.getenv("MONGO_URI"))
```

```
incidents_conn <- mongo(collection = "incidents", db = db_name,
                        url = Sys.getenv("MONGO_URI"))

# quick sanity check
reports_conn$count()
incidents_conn$count()
```

If `$count()` returns a number rather than an error, you're connected. This number is also your first data-volume signal.

6.4 The core mental model: pipelines are JSON, R is the courier

`$aggregate()` sends a pipeline to MongoDB's aggregation engine and gets back a data frame. The key thing to internalize: **the pipeline itself is not R code — it's JSON**. You're writing MongoDB's own query language as a string, and R is just the courier. That means:

- Field names are quoted strings ("report_number"), not R symbols.
- `$` in front of a name ("incident") means “the value of this field,” not an R accessor.
- The whole thing is an *array* (`[ ... ]`) of *stage objects* (`{ ... }`), executed top to bottom.

Think of a pipeline as an assembly line: each stage receives the documents from the stage before it, transforms them, and passes the result to the next stage. Nothing is “queried” all at once — it's a sequence of transformations.

```
pipeline <- '[ ... ]' # a JSON array, written as an R string

reports_with_incident <- reports_conn$aggregate(pipeline) |>
  tibble::as_tibble()
```

6.5 Don't download the whole collection

It's easy to fall into this pattern:

```
library(tidyverse)

report_cols <- '{
  "_id": 1, "report_number": 1, "title": 1, "plain_text": 1,
  "description": 1, "tags": 1, "authors": 1, "language": 1,
  "source_domain": 1, "date_published": 1
}'

reports <- reports_conn$find(fields = report_cols) |> as_tibble()

glimpse(reports)
```

Notice what this does: `$find()` with no `query` argument means “*give me every document in the collection.*” `fields` trims the columns, but it does nothing to trim the **rows** — you’re still pulling every document over the network and materializing it in R’s memory before you’ve filtered, grouped, or summarized anything. On a small collection this is harmless. On a collection with hundreds of thousands of documents (especially ones with a `plain_text` field, which tends to be large — or an `embedding` field, which is a full numeric vector per document), it’s the single most common source of slow, memory-hungry R/Mongo scripts.

The general rule: **MongoDB is a database with its own query and aggregation engine** — **use it as one**. Only bring rows into R once they’re already filtered, grouped, or reduced to roughly the size you actually need for the next step.

6.5.1 Filter rows on the server with query

`$find()` takes a `query` argument (the same JSON `$match`-style filter used in aggregation):

```
# Downloads everything, then filters in R - avoid this
reports <- reports_conn$find(fields = report_cols) |> as_tibble()
recent <- reports |> filter(date_published >= as.Date("2023-01-01"))

# Filters on the server, only recent documents cross the network
query <- '{ "date_published": { "$gte": { "$date": "2023-01-01T00:00:00Z" } } }'
recent <- reports_conn$find(query = query, fields = report_cols) |> as_tibble()

# Also useful: language filter, straight from mongolite's query syntax
reports_en <- reports_conn$find(
  query = '{"language": "en"}',
  fields = '{"_id": 1, "report_number": 1, "title": 1, "plain_text": 1}'
) |> as_tibble()
```

For the full query syntax `mongolite` supports, see <https://jeroen.github.io/mongolite/query-data.html#query-syntax>.

6.5.2 Let `$group` summarize before it ever reaches R

If your end goal is a summary (counts, averages, distinct values), don't pull raw rows into `dplyr::group_by() |> summarize()`. Do the equivalent with `$group` in an aggregation pipeline, so only the *summary* — not the underlying documents — travels over the wire:

```
pipeline <- '[
  { "$match":    { "language": "en" } },
  { "$group":    { "_id": "$source_domain", "n": { "$sum": 1 } } },
  { "$sort":     { "n": -1 } },
  { "$limit":    20 }
]'
```

```
top_domains <- reports_conn$aggregate(pipeline) |> as_tibble()
```

This never materializes the full `reports` collection anywhere — Mongo does the counting, and R receives 20 rows. The `date_published` year-grouping example below follows the same idea:

```
reports_per_year <- reports_conn$aggregate('[
  { "$group": {
    "_id": { "$substr": ["$date_published", 0, 4] },
    "n": { "$sum": 1 }
  } },
  { "$sort": { "_id": 1 } }
]') |> as_tibble()
```

6.5.3 Count before you fetch

Before running any `$find()/aggregate()` without a filter, get a sense of scale first:

```
reports_conn$count()           # whole collection
reports_conn$count('{ "language": "en" }')
```

If that number is larger than you're comfortable holding in memory (as a rough guide: tens of thousands of documents with large text fields), that's your signal to filter further, project fewer fields, or move to the batching approach in 8.5.

6.5.4 limit, skip, and \$sample for peeking or paging

```
# Peek at a handful of documents without downloading the rest
sample_reports <- reports_conn$find(fields = report_cols, limit = 100) |> as_tibble()

# Manual pagination
page <- reports_conn$find(fields = report_cols, limit = 500, skip = 1000) |> as_tibble()
```

For a genuinely random sample (not just “the first N”), use `$sample` in an aggregation pipeline instead of `limit`, since `find(limit = ...)` returns documents in storage order, not a random subset:

```
pipeline <- '[ { "$sample": { "size": 500 } } ]'
random_sample <- reports_conn$aggregate(pipeline) |> as_tibble()
```

6.5.5 Stream through a large result with \$iterate()

If you truly need to touch every document (e.g. something that can’t be expressed as an aggregation stage), avoid loading the entire result into a tibble at once. `mongolite` lets you get a cursor and pull batches:

```
cursor <- reports_conn$iterate(query = query, fields = report_cols)

repeat {
  batch <- cursor$batch(size = 1000)
  if (is.null(batch)) break

  # process this chunk only - e.g. write to disk, accumulate a summary,
  # or append to a growing file rather than a growing in-memory object
  batch |> as_tibble() |> process_batch()
}
```

This keeps memory usage bounded to one batch at a time, regardless of how large the underlying collection is.

6.5.6 A decision guide

| Your goal | Reach for |
|--|---|
| A quick look at the data's shape | <code>find(limit = ...)</code> or <code>\$sample</code> |
| Rows matching a condition | <code>find(query = ...)</code> or an aggregation <code>\$match</code> |
| A summary/aggregate statistic | An aggregation pipeline with <code>\$group</code> — never raw rows into <code>dplyr::summarize()</code> |
| A join across collections, summarized | <code>\$lookup</code> → <code>\$unwind</code> → <code>\$group</code> inside one pipeline |
| Every document, but too many to hold at once | <code>\$iterate()</code> with batching |
| “How big is this even going to be?” | <code>\$count()</code> first, always, before an unfiltered <code>\$find()/\$aggregate()</code> |

6.6 A worked pipeline, stage by stage: joining reports to incidents

This is the actual join you'll want for this schema — reports linked to the incident(s) that reference them by `report_number`.

```
pipeline <- '[
{
  "$lookup": {
    "from": "incidents",
    "localField": "report_number",
    "foreignField": "reports",
    "as": "incident"
  }
},
{
  "$unwind": {
    "path": "$incident",
    "preserveNullAndEmptyArrays": true
  }
},
{
  "$project": {
    "_id": 1,
    "report_number": 1,
    "title": 1,
    "plain_text": 1,
```

```

      "description": 1,
      "tags": 1,
      "authors": 1,
      "language": 1,
      "source_domain": 1,
      "date_published": 1,

      "incident_id": "$incident.incident_id",
      "incident_title": "$incident.title",
      "date": "$incident.date"
    }
  }
]'
```

```

reports_with_incident <- reports_conn$aggregate(pipeline) |>
  tibble::as_tibble()

```

6.6.1 Stage 1 — \$lookup (a SQL-style JOIN)

For every document in `reports`, MongoDB looks in the `incidents` collection for documents where `incidents.reports` matches this document's `report_number`. All matches are attached as an **array** under a new field called `incident`.

Mental model: *“For each report, go fetch related incidents and staple them on as a list.”*

6.6.2 Stage 2 — \$unwind (flatten the array)

`$lookup` always produces an array, even if there's exactly one match. `$unwind` explodes that array so each matched element becomes its own document (one report + one incident, instead of one report + an array of incidents). If a report matched 3 incidents, `$unwind` turns 1 document into 3.

`preserveNullAndEmptyArrays: true` is important — without it, reports with **no** matching incident would be silently dropped from the results. With it, they're kept, with `incident` set to `null`. Whether you want this depends on whether you're doing an inner join (drop unmatched) or a left join (keep unmatched) mentally.

6.6.3 Stage 3 — \$project (reshape the output)

- `"field": 1` means “keep this field, unchanged.”

- `"new_name": "$path.to.value"` means “create a field called `new_name`, populated from this path.” Since `$unwind` turned `incident` into a single embedded object (not an array), you can reach into it with dot notation (`$incident.incident_id`).

This last step is why the whole pipeline exists: `$lookup` + `$unwind` get the related data attached, and `$project` flattens it into a clean, tibble-friendly shape — no nested lists left for R to deal with.

6.7 The general thinking process for writing a new pipeline

Before writing any JSON, answer these in plain English:

1. **What’s my starting collection, and what’s the shape of one document in it?**
2. **What do I want one row of my final output to represent?** (One report? One report-incident pair? One tag with a count?)
3. **What has to happen to get from (1) to (2)?** — usually some combination of: bring in related data, filter out documents, reshape/rename fields, flatten arrays, or summarize groups.

Each of those verbs maps to a specific stage. Once you can state the transformation in English, the JSON is just translation:

| You want to... | Stage | Notes |
|--|---|--|
| Join in data from another collection | <code>\$lookup</code> | Always produces an array field |
| Flatten an array field (incl. <code>\$lookup</code> results) | <code>\$unwind</code> | Use <code>preserveNullAndEmptyArrays</code> deliberately |
| Filter documents (like SQL WHERE) | <code>\$match</code> | Put this as <i>early</i> as possible for performance |
| Choose/rename/compute output fields | <code>\$project</code> | 1 keeps, 0 excludes, “ <code>\$field</code> ” copies/renames |
| Add new fields without dropping others | <code>\$addFields</code> / <code>\$set</code> | Like <code>\$project</code> but additive — you don’t have to relist every field |
| Group and summarize (SQL GROUP BY) | <code>\$group</code> | <code>_id</code> is the grouping key; use <code>\$sum</code> , <code>\$avg</code> , <code>\$push</code> , etc. as accumulators |
| Sort results | <code>\$sort</code> | 1 ascending, -1 descending |
| Limit / skip (pagination) | <code>\$limit</code> , <code>\$skip</code> | Cheapest late in the pipeline, after filtering |
| Count documents | <code>\$count</code> | Collapses everything into { “count”: N } |

| You want to... | Stage | Notes |
|---|---|---|
| Random sample of documents | <code>\$sample</code> | Use instead of <code>limit</code> when you want a true random subset |
| Run several sub-pipelines and combine | <code>\$facet</code> | Useful for “give me totals AND a filtered list” in one round trip |
| Reshape a document to become one of its subfields | <code>\$replaceRoot</code> / <code>\$replaceWith</code> | Common after <code>\$unwind</code> when you only want the nested object |
| Traverse a hierarchy/graph (e.g. org charts) | <code>\$graphLookup</code> | Recursive <code>\$lookup</code> |

6.7.1 Worked example: `$match` + `$group` from scratch

Say you want: “For each *source_domain*, how many reports were published, restricted to reports published after 2020?”

Walk through the three questions:

1. Starting collection: `reports`, each document is one report.
2. Desired output row: one row per `source_domain`, with a count.
3. Transformation: filter by date, then group by domain and count.

```
pipeline <- '[
  {
    "$match": {
      "date_published": { "$gte": { "$date": "2020-01-01T00:00:00Z" } }
    }
  },
  {
    "$group": {
      "_id": "$source_domain",
      "n_reports": { "$sum": 1 }
    }
  },
  {
    "$sort": { "n_reports": -1 }
  }
]'
```

```
report_counts <- reports_conn$aggregate(pipeline) |> tibble::as_tibble()
```

Notice the pattern repeats: **filter/join stages first, reshape stages last**. `$match` should almost always come as early as possible (it reduces the document count before any expensive work), and `$project/$group/$sort` come afterward to shape the final result.

6.8 Two ways to join reports to incidents — pipeline vs. R-side

The example earlier does the join *inside MongoDB* with `$lookup`. You can also pull both collections separately and join them in R with `dplyr`, using `incidents$reports` (an array of `report_numbers`):

```
incidents <- incidents_conn$find(
  fields = '{
    "_id": 1, "incident_id": 1, "title": 1, "description": 1, "date": 1,
    "reports": 1, "Alleged deployer of AI system": 1,
    "Alleged developer of AI system": 1,
    "Alleged harmed or nearly harmed parties": 1
  }'
) |> as_tibble()

# one row per (incident_id, report_number) pair
incident_report_map <- incidents |>
  select(incident_id, reports) |>
  unnest_longer(reports) |>
  rename(report_number = reports)

# now every report row can carry its incident's metadata
reports_with_incident <- reports |>
  left_join(incident_report_map, by = "report_number") |>
  left_join(
    incidents |> select(incident_id, incident_title = title, date),
    by = "incident_id"
  )
```

Both approaches produce the same shape. Which one to reach for:

- **`$lookup/$unwind` in a pipeline** — join happens on the server, and only the already-joined, already-projected result crosses the network. Preferred when `reports` is large, or when you only need a subset of the joined result (combine with an early `$match`).

- **Pull + `unnest_longer()` + `left_join()` in R** — simpler to read and debug, **but as the data gets larger with tokenization** (creating a row for each token in plain text) handling all the data in memory is going to be costly.

6.9 Cleaning the text

`plain_text` is already stripped of most markup, but scraped news text still tends to carry boilerplate (bylines, “read more” links, share-button text). Do a light cleaning pass and keep the original column in case later analysis needs it.

Warning

The following is only a suggestion and is not guaranteed to produce fully cleaned text ready for analysis.

```
library(textclean)

reports <- reports_conn$find(fields = report_cols, limit = 100) |>
  as_tibble() |>
  mutate(plain_text = replace_na(plain_text, "")) |> # guard against NA before cleaning
  mutate(
    text_clean = plain_text |>
      replace_contraction() |> # "don't" -> "do not"
      replace_url() |>
      replace_html() |>
      str_squish() # trim/collapse whitespace
  )
```

6.10 Tokenising and analysing with `tidytext` — size-consciously

`tidytext` works on the “one-token-per-row” principle, which plugs directly into `dplyr/ggplot2`. Tokenizing multiplies your row count — one document becomes one row per word — so this is often where memory pressure actually shows up, even when the initial `find()/aggregate()` was fine. Narrow the input *before* tokenizing.

```

library(tidytext)

# Bad: tokenize the entire collection at once
# reports |> unnest_tokens(word, plain_text)

# Better: filter server-side to what the analysis actually needs, and
# project away fields (like description, embedding) you don't need here
query <- '{ "language": "en", "date_published": { "$gte": { "$date": "2024-01-01T00:00:00Z" } } }'
subset_cols <- '{ "_id": 1, "report_number": 1, "plain_text": 1 }'

reports_subset <- reports_conn$find(query = query, fields = subset_cols) |>
  as_tibble() |>
  mutate(plain_text = replace_na(plain_text, ""))

tokens <- reports_subset |>
  select(report_number, plain_text) |>
  unnest_tokens(word, plain_text) |>
  anti_join(stop_words, by = "word")

```

For exploratory work (trying out a TF-IDF approach, testing a topic model), work on a `$sample` subset first, and only run the full-size version once the approach is validated, the same instinct as testing a pipeline stage-by-stage.

6.10.1 Word frequencies

```

top_words <- tokens |>
  count(word, sort = TRUE)

top_words |>
  slice_max(n, n = 20) |>
  ggplot(aes(x = reorder(word, n), y = n)) +
  geom_col() +
  coord_flip() +
  labs(x = NULL, y = "count", title = "Most frequent words across reports")

```

6.10.2 TF-IDF (which words distinguish one incident/domain from another)

Use `incident_id` or `source_domain` as the grouping variable to see what language is distinctive to a given incident or outlet:

```
tfidf <- reports_with_incident |>
  select(report_number, incident_id, text_clean) |>
  unnest_tokens(word, text_clean) |>
  anti_join(stop_words, by = "word") |>
  count(incident_id, word, sort = TRUE) |>
  bind_tf_idf(word, incident_id, n)

tfidf |>
  group_by(incident_id) |>
  slice_max(tf_idf, n = 10) |>
  ungroup()
```

6.10.3 N-grams

For phrase-level patterns rather than single words:

```
bigrams <- reports |>
  unnest_tokens(bigram, text_clean, token = "ngrams", n = 2) |>
  separate(bigram, into = c("word1", "word2"), sep = " ") |>
  filter(!word1 %in% stop_words$word, !word2 %in% stop_words$word) |>
  count(word1, word2, sort = TRUE)
```

6.10.4 Summarize text metrics server-side instead of tokenizing in R

If the *final* thing you want is a summary statistic derived from text — not the text content itself — MongoDB’s own operators can sometimes get you there without tokenizing in R at all. For example, a rough word count per document with `$size` and `$split`:

```
pipeline <- '[
  {
    "$addFields": {
      "word_count": { "$size": { "$split": ["$plain_text", " "] } }
    }
  },
  { "$project": { "report_number": 1, "word_count": 1 } }
]'
```

```
word_counts <- reports_conn$aggregate(pipeline) |> as_tibble()
```

This computes a number per document server-side and returns one row per document — never the text itself.

6.10.5 Process the full corpus in chunks, when you must

If a text-mining task genuinely needs the whole corpus (e.g. a full-collection document-term matrix), combine `$iterate()` batching with incremental summarizing instead of binding every batch into one giant tibble:

```
cursor <- reports_conn$iterate(query = query, fields = subset_cols)
word_freq <- list()

repeat {
  batch <- cursor$batch(size = 500)
  if (is.null(batch)) break

  batch_freq <- batch |>
    as_tibble() |>
    mutate(plain_text = replace_na(plain_text, "")) |>
    unnest_tokens(word, plain_text) |>
    anti_join(stop_words, by = "word") |>
    count(word)

  word_freq[[length(word_freq) + 1]] <- batch_freq
}

# Combine only the small, already-summarized per-batch counts
total_word_freq <- bind_rows(word_freq) |>
  group_by(word) |>
  summarize(n = sum(n), .groups = "drop") |>
  arrange(desc(n))
```

The corpus itself is never fully resident in memory — only one batch, plus a running (and much smaller) frequency table.

6.11 Sentiment analysis

`tidytext` bundles several sentiment lexicons (`bing`, `afinn`, `nrc`, `loughran`). Join tokens to a lexicon the same way you joined stop words. `loughran` is worth knowing about specifically — it was built for financial/news text and includes categories like “uncertainty” and “litigious”, which can be more informative than generic positive/negative for news-style incident reports.

```
sentiment_scores <- tokens |>
  inner_join(get_sentiments("bing"), by = "word") |>
  count(report_number, sentiment) |>
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) |>
  mutate(net_sentiment = positive - negative)
```

For a numeric score per word (rather than positive/negative labels), use `afinn`:

```
afinn_scores <- tokens |>
  inner_join(get_sentiments("afinn"), by = "word") |>
  group_by(report_number) |>
  summarise(sentiment_score = sum(value), .groups = "drop")
```

`get_sentiments()` will prompt a one-time download/license confirmation the first time you use a given lexicon — do this well before you need it, in case you don't have internet access when it matters (e.g. at a competition venue).

6.12 Word co-occurrence / correlation (optional, `widyr`)

Useful for exploring which words tend to appear together within the same report — do this on a `count()`-reduced or sampled set of tokens, not the raw full-corpus token table, since pairwise operations grow fast:

```
library(widyr)

word_pairs <- tokens |>
  pairwise_count(word, report_number, sort = TRUE)

word_cors <- tokens |>
  group_by(word) |>
  filter(n() >= 20) |> # drop very rare words first
  pairwise_cor(word, report_number, sort = TRUE)
```

6.13 Debugging and building pipelines incrementally

- **Build incrementally.** Write one stage, run `$aggregate()`, inspect the output, *then* add the next stage. Debugging a 5-stage pipeline all at once is much harder than debugging it one stage at a time.
- **Comment out stages by truncating the array.** Since the pipeline is just a string, you can test `'[ stage1 ]'`, then `'[ stage1, stage2 ]'`, and so on.
- **Watch for the array-vs-object trap.** After `$lookup`, a field is an array. After `$unwind`, that same field is a single object. Referencing `$incident.title` only works *after* unwinding (or if you know there's exactly one match and use `$arrayElemAt`).
- **Use `jsonlite::toJSON()/fromJSON()` to build pipelines as R lists** instead of raw strings, if you find quoting JSON by hand error-prone:

```
pipeline_list <- list(  
  list(`$match` = list(date_published = list(`$gte` = "2020-01-01"))),  
  list(`$group` = list(`_id` = "$source_domain", n_reports = list(`$sum` = 1)))  
)  
pipeline <- jsonlite::toJSON(pipeline_list, auto_unbox = TRUE)
```

This trades JSON-string fiddliness for R's backtick-quoted-name syntax (``$match``) — pick whichever you find less error-prone. Many R/Mongo users stick with raw JSON strings for anything beyond 2–3 stages, since MongoDB's own documentation and the shell both use JSON, so the translation from “docs example” to “your pipeline” stays literal.

- **`preserveNullAndEmptyArrays` is a decision, not a default.** Always ask explicitly: do I want to drop documents with no match, or keep them with `null`?
- **Field order in `$project` doesn't matter to Mongo**, but keeping it consistent with your intended output column order makes the resulting tibble easier to read.
- **Test aggregations in MongoDB Compass or `mongosh` first**, if you have access — it's faster to iterate there and then paste the finished JSON into your R string than to round-trip through R for every tweak.

6.14 Putting it together: a minimal end-to-end script

```

library(mongolite)
library(tidyverse)
library(tidytext)
library(textclean)

db_name <- "aiidprod"
reports_conn <- mongo(collection = "reports", db = db_name,
                      url = Sys.getenv("MONGO_URI"))

reports <- reports_conn$find(
  fields = '{"_id": 1, "report_number": 1, "title": 1, "plain_text": 1, "tags": 1}'
) |>
  as_tibble() |>
  mutate(
    plain_text = replace_na(plain_text, ""),
    text_clean = str_squish(replace_contraction(replace_url(plain_text, "")))
  )

word_counts <- reports |>
  unnest_tokens(word, text_clean) |>
  anti_join(stop_words, by = "word") |>
  count(word, sort = TRUE)

word_counts |> slice_max(n, n = 15)

```

6.15 Common pitfalls

- **Connection refused / timeout** — your IP isn't allow-listed in Atlas Network Access, or you're on a network (uni wifi, VPN) that blocks the MongoDB port. Test the connection *before* you need it.
- **Pulling embedding by accident** — it's a full numeric vector per document; leave it out of your fields projection unless you specifically need it, or your `find()` calls will be slow and memory-heavy.
- **Using text instead of plain_text** — `text` can retain markdown/HTML from the original source, which will pollute your tokens (`<`, `##`, `href=`, etc.). Prefer `plain_text` for tokenising.
- **unnest_wider() on the wrong field** — `authors`, `tags`, and the “alleged parties” fields are arrays of *strings*, so they need `unnest_longer()`. `unnest_wider()` is for fields that are nested *objects* (there aren't many of those in `reports/incidents`).

- **Joining incidents to reports on the wrong key** — `incidents.reports` holds `report_number` values, not `_id` (objectId) values. Join on `report_number`, not `_id`.
- **Encoding issues (garbled characters)** — check `Encoding()` on your text column and re-encode with `iconv()` if you see mojibake.
- **Credentials in scripts** — always use `Sys.getenv()` against an `.Renviron` file, never paste the connection string directly into a script you'll share or commit.
- **Downloading before filtering/summarizing** — the most common source of slow, memory-heavy scripts against this collection specifically; before writing an unfiltered `find()/aggregate()`.

6.16 Quick reference

6.16.1 Pipeline stages

The short version: `$match/$lookup` early, `$project/$group/$sort/$limit` late.

6.16.2 Package cheat sheet

| Task | Package | Key function(s) |
|------------------------|---|---|
| Connect to MongoDB | <code>mongolite</code> | <code>mongo()</code> , <code>\$find()</code> , <code>\$aggregate()</code> , <code>\$count()</code> , <code>\$iterate()</code> |
| Wrangle data | <code>dplyr</code> / <code>tidyr</code> | <code>mutate()</code> , <code>filter()</code> , <code>unnest_longer()</code> , <code>left_join()</code> |
| Clean text | <code>textclean</code> | <code>replace_contraction()</code> , <code>replace_url()</code> , <code>replace_html()</code> |
| Tokenise | <code>tidytext</code> | <code>unnest_tokens()</code> |
| Frequency / TF-IDF | <code>tidytext</code> | <code>count()</code> , <code>bind_tf_idf()</code> |
| Sentiment | <code>tidytext</code> | <code>get_sentiments()</code> , <code>inner_join()</code> |
| Word pairs/correlation | <code>widyr</code> | <code>pairwise_count()</code> , <code>pairwise_cor()</code> |
| Plotting | <code>ggplot2</code> | <code>geom_col()</code> , <code>geom_point()</code> |

6.17 Summary checklists

Writing any new pipeline

1. State in one sentence what one output row should represent.
2. List the stages needed to get there, in order (join → filter → flatten → reshape → group → sort → limit).
3. Write and test one stage at a time.
4. Put `$match` as early as possible; put `$sort/$limit` as late as possible.
5. After any `$lookup`, decide immediately whether you need `$unwind`, and whether unmatched documents should be kept.
6. Finish with `$project` (or `$set`) so the shape handed to `tibble::as_tibble()` is already flat and clean.
7. Before any unfiltered `find()/aggregate()`, run `$count()` first.

Size-conscious tidyverse/text-mining work

1. Ask what the *smallest* result is that answers your question — a count? a top-20 list? — and try to get MongoDB to produce that directly.
2. Exclude large fields (`plain_text`, `description`, `embedding`) from `fields/$project` unless this specific step needs them.
3. Filter (`query/$match`) and, for exploration, sample (`$sample`) before tokenizing.
4. Prefer `$group/$addFields` for summaries over pulling raw rows into `dplyr::summarize()` or `unnest_tokens()`.
5. If you must touch the full corpus, batch with `$iterate()` and combine only small, already-summarized pieces — never accumulate raw batches into one growing tibble.

Good luck with the competition.

7 Data dictionary for AI Incident Database

Accuracy of data dictionary

This is not an official data dictionary provided by the AI Incident Database. There might be conflicts or issues with the actual database schema with online resources.

7.1 Overview

At a high level, the collections fall into three groups:

- **Core record collections:** `incidents`, `reports`, `entities`, `entity_relationships` are the canonical, published data users browse in the “Discover” app.
- **Classification collections:** `taxa`, `classifications` are the flexible, namespaced taxonomy system used to tag incidents/reports (e.g. the CSET taxonomy).
- **Workflow / pipeline collections:** `submissions`, `candidates` `quickadd`, `duplicates`, `entity_duplicates`, `incident_links`, `checklists`, `migrations` are there to support the editorial pipeline that turns a scraped news article into a published incident, plus ancillary tools.

Each collection section below lists field name, inferred type, and a description. Types marked `array<T>` denote an array whose elements are of type `T`; a bare `array` means an empty array was observed and no element type could be inferred. `object{...}` denotes an embedded sub-document, with its own fields described inline or in a nested table.

7.2 Core record collections

7.2.1 incidents

The canonical, published incident records which are the primary entity of the database. Each incident aggregates one or more `reports`.

| Field | Type | Description |
|---|---------------|--|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>incident_id</code> | number | Public, human-facing incident number (used in URLs like <code>/cite/123</code>). |
| <code>title</code> | string | Short incident title shown in listings. |
| <code>description</code> | string | Editor-written summary of the incident. |
| <code>date</code> | string | Date the incident occurred (YYYY-MM-DD). |
| <code>reports</code> | array<number> | <code>report_numbers</code> of the reports that make up this incident. |
| <code>editors</code> | array<string> | Identifiers/usernames of editors who have contributed to the record. |
| <code>editor_notes</code> | string | Free-text internal notes for editors. |
| <code>implicated_systems</code> | array<string> | <code>entity_ids</code> of the AI system(s) implicated. |
| <code>Alleged
deployer of AI
system /
AllegedDeployerOfAISystem</code> | array<string> | <code>entity_ids</code> of the party alleged to have deployed the system. Both the legacy space-separated key and a camelCase duplicate are present; the camelCase versions were added because MongoDB Atlas App Services/Realm (<code>_id__baas_transaction</code> , see below) cannot use field names containing spaces in GraphQL. |
| <code>Alleged
developer of
AI system /
AllegedDeveloperOfAISystem</code> | array<string> | <code>entity_ids</code> of the party alleged to have developed the system (same legacy/camelCase pairing as above). |
| <code>Alleged harmed
or nearly
harmed parties
/
AllegedHarmedOrNearlyHarmedParties</code> | array<string> | <code>entity_ids</code> of parties alleged to have been harmed or nearly harmed. |
| <code>nlp_similar_incidents</code> | array<object> | NLP-generated recommendations of related incidents: <code>{incident_id, similarity}</code> . |
| <code>editor_similar_incidents</code> | array<number> | Incident IDs an editor has manually confirmed as similar. |
| <code>editor_dissimilar_incidents</code> | array<number> | Incident IDs an editor has manually marked as <i>not</i> similar (overrides NLP suggestions). |
| <code>flagged_dissimilar_incidents</code> | array<number> | Incident IDs flagged by users/editors for review as likely-incorrect similarity matches. |

| Field | Type | Description |
|---------------------------------------|--|--|
| <code>embedding</code> | object
 <code>{report_numbers, from_reports}</code> | Text embedding vector representing the incident, plus the <code>report_numbers</code> it was derived from. Powers similarity search from <code>reports</code> in the spatial view. |
| <code>tsne</code> | object
<code>{x, y}</code> | 2-D t-SNE projection of the embedding, used to plot the incident in the “Spatial View” visualization. |
| <code>created_at</code> | date | Record creation timestamp. |
| <code>date_modified</code> | date | Last-modified timestamp. |
| <code>epoch_date_modified</code> | number | <code>date_modified</code> as a Unix epoch, kept for fast sorting/filtering. |
| <code>_id__baas_transaction_id</code> | objectId | Legacy transaction identifier from MongoDB Atlas App Services (formerly “Realm”, internally “baas”), used by trigger functions; not part of the incident’s semantic data. |

7.2.2 reports

Individual news articles / source documents. A report can be linked to zero (candidate), one, or more incidents via `incidents.reports`.

| Field | Type | Description |
|-----------------------------------|---------------|--|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>report_number</code> | number | Public, human-facing report number. |
| <code>title</code> | string | Article/report title. |
| <code>text</code> | string | Full report text (may include markdown/HTML from ingestion). |
| <code>plain_text</code> | string | Plain-text version of <code>text</code> , stripped of markup. |
| <code>description</code> | string | Short editor- or auto-generated summary. |
| <code>url</code> | string | Source URL of the report. |
| <code>source_domain</code> | string | Domain the report was published on (e.g. <code>nytimes.com</code>). |
| <code>image_url</code> | string | URL of a representative image for the report. |
| <code>clouidary_id</code> | string | Identifier of the image as hosted on Clouidary. |
| <code>authors</code> | array<string> | Byline authors of the source article. |
| <code>submitters</code> | array<string> | AIID users who submitted this report. |
| <code>language</code> | string | ISO language code of the report text. |
| <code>tags</code> | array | Free-form tags applied to the report. |
| <code>date_published</code> | date | Date the source article was published. |
| <code>date_submitted</code> | date | Date the report was submitted to AIID. |
| <code>date_downloaded</code> | date | Date the report content was scraped/archived. |
| <code>date_modified</code> | date | Last-modified timestamp. |
| <code>epoch_date_published</code> | number | <code>date_published</code> as a Unix epoch. |
| <code>epoch_date_submitted</code> | number | <code>date_submitted</code> as a Unix epoch. |

| Field | Type | Description |
|-------------------------------|--|---|
| embedding | object
{ vector ,
from_text_hash } | Text embedding vector plus a hash of the source text it was computed from (used to detect when re-embedding is needed after edits). |
| is_incident_report | boolean | Whether the report is attached to a published incident (false for background/context articles). |
| flag | boolean | Whether the report has been flagged for editorial review. |
| snippet_max_characters | integer | Display truncation length used when rendering a snippet/preview of this report. |
| editor_notes | string | Free-text internal editor notes. |
| created_at | date | Record creation timestamp. |
| inputs_outputs | — | Reserved/unused in this export (always null). |
| quiet | — | Reserved/unused in this export (always null); elsewhere used to suppress notification side-effects on save. |

7.2.3 entities

The controlled list of named parties (developers, deployers, harmed parties, AI systems) referenced across incidents.

| Field | Type | Description |
|----------------------|----------|---|
| _id | objectId | MongoDB primary key. |
| entity_id | string | Slug used as the stable reference key (e.g. in incidents.implicated_systems). |
| name | string | Display name of the entity. |
| created_at | date | Record creation timestamp (null for entities created before this field was introduced). |
| date_modified | date | Last-modified timestamp. |

7.2.4 entity_relationships

Subject–predicate–object triples linking entities (e.g. “Company A” *is a subsidiary of* “Company B”).

| Field | Type | Description |
|---------------------------|----------|---|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>sub</code> | string | <code>entity_id</code> of the subject. |
| <code>pred</code> | string | Relationship/predicate name. |
| <code>obj</code> | string | <code>entity_id</code> of the object. |
| <code>is_symmetric</code> | boolean | Whether the relationship applies equally in both directions (so a single row implies the reverse triple too). |

7.3 Classification (taxonomy) collections

7.3.1 taxa

Defines each classification taxonomy/namespace (e.g. the CSET taxonomy) and the schema of its fields — effectively metadata describing the “columns” available in `classifications.attributes` for that namespace.

| Field | Type | Description |
|---------------------------------------|---------------|---|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>namespace</code> | string | Unique taxonomy identifier (referenced by <code>classifications.namespace</code>). |
| <code>description</code> | string | Description of the taxonomy shown to users. |
| <code>weight</code> | number | Sort order among taxonomies. |
| <code>complete_entities</code> | boolean | Whether classifiers must fully specify implicated entities for this taxonomy. |
| <code>automatedClassifications</code> | boolean | Whether classifications for this taxonomy can be generated automatically (e.g. by a model) rather than only by human editors. |
| <code>field_list</code> | array<object> | The taxonomy’s field definitions — one entry per classifiable attribute. See below. |
| <code>dummy_fields</code> | array<object> | Template/placeholder field definitions used as a starting point when authoring a new taxonomy field. |

field_list / dummy_fields element schema (each entry describes one attribute of the taxonomy):

| Sub-field | Type | Description |
|---|---------|--|
| <code>field_number</code> | string | Stable identifier for the field within the taxonomy. |
| <code>short_name</code> | string | Machine-friendly key, referenced by <code>classifications.attributes[].short_name</code> . |
| <code>long_name</code> | string | Human-readable label. |
| <code>short_description / long_description</code> | string | Help text of varying length shown to classifiers. |
| <code>display_type</code> | string | UI widget used to capture the value (e.g. text, select, multi-select). |
| <code>mongo_type</code> | string | Underlying MongoDB type the value is stored as. |
| <code>default</code> | string | Default value when a new classification is created. |
| <code>placeholder</code> | string | Placeholder text shown in the input widget. |
| <code>permitted_values</code> | array | Allowed values, for enumerated/select fields. |
| <code>weight</code> | number | Sort order among fields within the taxonomy. |
| <code>instant_facet</code> | boolean | Whether the field is exposed as an instant-search facet in Discover. |
| <code>required</code> | boolean | Whether a value must be supplied. |
| <code>public</code> | boolean | Whether the field is visible to non-editor users. |
| <code>hide_search</code> | boolean | Whether the field is excluded from search indexing. |
| <code>notes</code> | string | Internal notes about the field. |

7.3.2 classifications

The actual tag/attribute values applied to one or more incidents and reports under a given taxonomy namespace.

| Field | Type | Description |
|---------------------------|---------------|---|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>namespace</code> | string | Taxonomy this classification belongs to (references <code>taxa.namespace</code>). |
| <code>incident_ids</code> | array<number> | <code>incident_ids</code> this classification applies to. |
| <code>reports</code> | array | Report(s) used as evidence for the classification. |
| <code>attributes</code> | array<object> | The tagged values: <code>{short_name, value_json}</code> , where <code>short_name</code> matches a <code>taxa.field_list[].short_name</code> and <code>value_json</code> holds the (JSON-encoded) value for that field. |
| <code>published</code> | boolean | Whether the classification is publicly visible. |

| Field | Type | Description |
|------------|--------|---|
| notes | string | Free-text notes about the classification. |
| created_at | date | Record creation timestamp. |

7.4 Editorial pipeline & workflow collections

7.4.1 submissions

User-submitted candidate reports/incidents awaiting editorial review before being promoted into reports/incidents.

| Field | Type | Description |
|--|---------------|--|
| _id | objectId | MongoDB primary key. |
| status | string | Workflow status of the submission (e.g. pending review). |
| user | string | Identifier of the submitting user. |
| title | string | Proposed report title. |
| text / plain_text | string | Report body (raw and stripped). |
| description | string | Proposed summary. |
| url | string | Source URL. |
| source_domain | string | Domain of the source URL. |
| image_url / cloudinary_id | string | Representative image and its Cloudinary asset id. |
| language | string | ISO language code. |
| authors / submitters | array<string> | Byline authors / AIID users who submitted. |
| tags | array | Free-form tags. |
| date_published /
date_submitted /
date_downloaded /
date_modified | string | Lifecycle dates, stored as strings pending promotion to typed dates in reports . |
| incident_date | string | Proposed date of the incident itself. |
| incident_title | — | Proposed incident title, if creating a new incident (null until set). |
| incident_ids | array<number> | Existing incident(s) the submitter believes this report belongs to. |
| deployers / developers /
harmed_parties | array<string> | Proposed alleged deployer/developer/harmed-party entities. |
| implicated_systems | array<string> | Proposed implicated AI system entities. |

| Field | Type | Description |
|--|--|---|
| <code>incident_editors</code> | <code>array<string></code> | Proposed editors for the resulting incident. |
| <code>nlp_similar_incidents</code> | <code>array<object></code> | NLP-suggested similar incidents: <code>{incident_id, similarity}</code> . |
| <code>editor_similar_incidents</code> / <code>editor_dissimilar_incidents</code> | — | Editor overrides of the NLP suggestion (null until an editor acts on the submission). |
| <code>embedding</code> | <code>object {vector, from_text_hash}</code> | Text embedding and source-text hash, as in <code>reports</code> . |
| <code>snippet_max_characters</code> | <code>number</code> | Snippet truncation length for preview. |
| <code>editor_notes</code> | <code>string</code> | Internal notes. |
| <code>quiet</code> | <code>boolean</code> | Whether to suppress notifications when the submission is saved/updated. |
| <code>created_at</code> | <code>date</code> | Record creation timestamp. |

7.4.2 candidates

Articles discovered automatically (e.g. via scraping/NLP monitoring) that *may* describe an AI incident, awaiting triage before becoming a `submission`. Distinct from `submissions`, which are user-initiated.

| Field | Type | Description |
|--|--|---|
| <code>_id</code> | <code>objectId</code> | MongoDB primary key. |
| <code>title</code> / <code>text</code> / <code>plain_text</code> | <code>string</code> | Article content, as scraped. |
| <code>url</code> | <code>string</code> | Source URL. |
| <code>date_published</code> | <code>string</code> | Publish date as extracted from the source. |
| <code>date_scraped</code> | <code>string</code> | Date the scraper collected the article. |
| <code>matching_keywords</code> | <code>array<string></code> | Keywords that caused the scraper to surface this article. |
| <code>matching_harm_keywords</code> | <code>array<string></code> | Harm-specific keywords matched. |
| <code>matching_entities</code> | <code>array<string></code> | Known AIID entities detected in the text. |
| <code>embedding</code> | <code>object {vector, from_text_hash}</code> | Text embedding and source-text hash. |
| <code>similarity</code> | <code>number</code> | Similarity score to existing incidents, used to rank candidates. |
| <code>classification_similarity</code> | <code>array<object></code> | Per-taxonomy match likelihoods: <code>{classification, similarity}</code> . |

| Field | Type | Description |
|--------------------|---------|--|
| <code>match</code> | boolean | Whether the candidate has been confirmed as a genuine (likely) incident match. |

7.4.3 quickadd

Lightweight bookmarks of a URL for later review — the fastest path for a user to flag a link without filling out a full submission.

| Field | Type | Description |
|-----------------------------|----------|---|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>url</code> | string | Bookmarked URL. |
| <code>source_domain</code> | string | Domain of the URL. |
| <code>incident_id</code> | number | Incident the link is proposed to relate to, if any. |
| <code>date_submitted</code> | string | Date the link was quick-added. |
| <code>created_at</code> | date | Record creation timestamp. |

7.4.4 duplicates

Maps an incident record that was identified as a duplicate onto the canonical incident it was merged into.

| Field | Type | Description |
|--|----------|--|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>duplicate_incident_number</code> | number | The incident number that is a duplicate. |
| <code>true_incident_number</code> | number | The canonical incident number it should be treated as. |
| <code>created_at</code> | date | Record creation timestamp. |

7.4.5 entity_duplicates

Same merge pattern as `duplicates`, for entities.

| Field | Type | Description |
|----------------------------------|----------|--|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>duplicate_entity_id</code> | string | Entity id that is a duplicate. |
| <code>true_entity_id</code> | string | Canonical entity id it should be treated as. |

7.4.6 incident_links

Cross-references between an AIID incident and the same event as recorded in another external database (e.g. the OECD AI Incidents Monitor).

| Field | Type | Description |
|-------------------------------|----------|---|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>incident_id</code> | number | AIID incident number. |
| <code>source_namespace</code> | string | Identifier of the external database/source. |
| <code>sameAs</code> | string | URI or identifier of the equivalent record in that external source. |

7.4.7 checklists

User-authored AI risk-assessment checklists, built against the incident taxonomy to help teams evaluate a system they are building or deploying. Independent of any single incident.

| Field | Type | Description |
|---|---------------|---|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>id</code> | string | Checklist identifier. |
| <code>owner_id</code> | string | User who owns the checklist. |
| <code>name</code> | string | Checklist title. |
| <code>about</code> | string | Description of the system/project being assessed. |
| <code>tags_goals</code> /
<code>tags_methods</code> /
<code>tags_other</code> | array<string> | Free-form tags describing the goals, methods, and other attributes of the system. |
| <code>risks</code> | array<object> | The individual risk entries; see below. |
| <code>date_created</code> /
<code>date_updated</code> /
<code>created_at</code> | date | Lifecycle timestamps. |

risks element schema:

| Sub-field | Type | Description |
|--------------------|--------|--|
| <code>id</code> | string | Risk entry identifier. |
| <code>title</code> | string | Short label for the risk. |
| <code>tags</code> | array | Taxonomy tags the risk was derived from. |

| Sub-field | Type | Description |
|--------------------------|---------------|--|
| <code>precedents</code> | array | Related past incidents cited as precedent for the risk. |
| <code>likelihood</code> | string/number | Assessed likelihood of the risk occurring. |
| <code>severity</code> | string/number | Assessed severity if it occurs. |
| <code>risk_status</code> | string | Current mitigation/review status. |
| <code>risk_notes</code> | string | Free-text notes. |
| <code>generated</code> | boolean | Whether the entry was auto-generated from the taxonomy vs. added manually. |
| <code>touched</code> | boolean | Whether the user has reviewed/edited the auto-generated entry. |

7.4.8 migrations

Internal bookkeeping of applied database schema migrations; not part of the domain model.

| Field | Type | Description |
|--|----------|---|
| <code>_id</code> | objectId | MongoDB primary key. |
| <code>migrationName</code> | string | Name/identifier of the migration script. |
| <code>createdAt</code> / <code>created_at</code> | date | When the migration was applied (both a legacy and current timestamp field are present). |

7.5 Notes on schema conventions

- **Legacy/camelCase field pairs.** A few `incidents` fields exist twice — once with spaces (e.g. `Alleged deployer of AI system`) and once in camelCase (`AllegedDeployerOfAISystem`). The camelCase versions exist because MongoDB Atlas App Services' GraphQL layer cannot expose field names containing spaces; both are kept in sync for backward compatibility.
- **array with no element type.** Where the scan reports a bare `array` alongside a more specific `array<T>` for the same field, it simply means some documents had an empty array for that field, so no element type could be inferred from them.
- **Embeddings.** `embedding.vector` fields store dense vector representations of text (incident/report/candidate content) used for nearest-neighbor similarity search and clustering (e.g. `tsne` for 2-D visualization). `from_text_hash` / `from_reports` record what the embedding was computed from, so it can be invalidated and recomputed when the source text changes.

- **Namespacing.** The taxonomy system (**taxa + classifications**) is deliberately pluralistic: multiple organizations can define their own **namespace** (e.g. a general taxonomy alongside domain-specific ones) and apply independent classifications to the same incidents, rather than the database enforcing one canonical taxonomy.

8 Resources for tidytext (Text Mining with R)

i Why this matters

The dataset is mostly free text. Chapter 6 already walks through tokenising, TF-IDF, sentiment, and word co-occurrence — this chapter is where to go for more depth or techniques it doesn't cover.

8.1 The core reference

- [Text Mining with R](#) by Julia Silge and David Robinson (free, by the tidytext package authors). Relevant chapters:
 - Ch. 1 — the tidy text format (`unnest_tokens()`)
 - Ch. 2 — sentiment analysis
 - Ch. 3 — word/document frequency (tf-idf)
 - Ch. 4 — n-grams and word correlations
 - Ch. 6 — topic modeling
- `?tidytext::unnest_tokens`, `?tidytext::bind_tf_idf`, or `vignette(package = "tidytext")` for the function reference.

8.2 If you want to find latent themes: topic modeling (LDA)

```
library(topicmodels)

dtm <- tokens_clean |>
  count(report_number, word) |>
  cast_dtm(report_number, word, n)

lda_model <- LDA(dtm, k = 5, control = list(seed = 1234))

tidy(lda_model, matrix = "beta") |> # word probabilities per topic
```

```
group_by(topic) |>
slice_max(beta, n = 10)
```

`tidy()` converts the model back into a tidy data frame so the rest of your `dplyr/ggplot2` pipeline still applies. See Text Mining with R, Ch. 6.

8.3 If word variants are diluting your counts: stemming

```
library(SnowballC)

tokens_clean <- tokens_clean |>
  mutate(word_stem = wordStem(word))
```

Use deliberately, not automatically — stemming can hurt readability in a report meant for human judges.

8.4 If you want more corpus-linguistics tooling: quanteda

[quanteda](#) is an alternative to `tidytext`, with built-in keyword-in-context (`kwic()`) and document-feature matrix tools. Pick one framework and stay consistent through your analysis.

8.5 Visualizing results

- `ggplot2::geom_col()` / `geom_point()` for frequency and tf-idf charts.
- `wordcloud2` for a quick visual summary — pair with an actual frequency table, since word clouds are weak for precise comparisons.
- `ggraph` + `igraph` on `widyr::pairwise_cor()` output for word-correlation networks.

8.6 Cheat sheet

Posit's [tidytext cheat sheet](#) — one page, covers tokenisation, sentiment, tf-idf. Keep it open during the event.

8.7 Keep it in scope

Text mining is a means to an analytical insight, not the insight itself. A clear tf-idf or sentiment analysis tied to a specific question scores better than a kitchen-sink pipeline with no narrative tying it together.

9 Good Visualisation & Tables

i Why this matters

Analysis & Evidence (35 points) and Report Clarity & Storytelling (20 points) — more than half of your score comes from these elements, so it's worth looking over these materials to create good visualisation and tables. Even solid analysis loses marks if it's shown as raw R output or a cluttered default chart.

Start from the message, not the chart: write the one-sentence finding first, then pick the plot or table that makes it obvious. A few things worth knowing regardless of which chart you pick, position and length (bar charts, scatter plots) are judged far more accurately than angle and area (pie charts), so prefer the former; sort categories by value instead of leaving them alphabetical; and for tables, don't paste a `summary()` printout into the report, `broom::tidy()` turns a fitted model into a data frame, and `gtsummary::tbl_regression()` turns that into a table you can actually trim down to what answers your question.

For the full treatment, perception, colour palettes, a worked diagnose/redesign/evaluate example, and table-formatting details, go through the two Monash ETC5523 (Communicating with Data) lectures this is drawn from:

- [Week 5 — from model outputs to communicative tables](#)
- [Week 6 — effective data visualisation](#)